

Microservices Implementation Roadmap (Local First → AWS Later)

This roadmap is designed so you can proceed incrementally.

How you'll work with me

- When you say “**Implement Step X**”, I will implement **only that step**.
 - You can pause for days and resume by requesting the next step.
-

Step 0 — Local prerequisites

Goal - Get a stable local baseline for development.

Do - Decide versions (recommended: Spring Boot 3.x + Spring Cloud 2023.x). - Create a multi-module repo layout (or separate repos) for: - catalog-service - order-service - api-gateway - discovery-server - config-server - Start MySQL locally (Docker) with 2 schemas/DBs: - catalog_db - order_db

Exit criteria - MySQL is running locally. - You can connect to both DBs from your laptop.

Step 1 — Build and test both microservice APIs directly (no gateway)

Goal - Prove each service works independently.

Do - Implement catalog-service REST endpoints and connect to catalog_db. - Implement order-service REST endpoints and connect to order_db. - Add: - validation - consistent error responses - health endpoints

How to test - Call both services directly using their ports. - Verify DB writes/reads.

Exit criteria - Both services run locally and pass your basic Postman/curl tests.

Step 2 — Add service-to-service call (order → catalog) + resilience baseline

Goal - order-service can validate/enrich using catalog-service reliably.

Do - Implement outbound client from order-service to catalog-service. - Add: - timeouts - retry policy - circuit breaker + fallback - bulkhead - Add idempotency strategy for order creation.

How to test - Stop catalog-service and verify order-service fails fast / falls back (no hangs).

Exit criteria - order-service remains responsive even when catalog-service is down/slow.

Step 3 — Add API Gateway (routing only)

Goal - Access both services through a single entry point.

Do - Add api-gateway. - Configure routes: - /catalog/** → catalog-service - /orders/** → order-service

How to test - Run gateway + services; call APIs only via gateway.

Exit criteria - All endpoints work through gateway (same behavior as direct calls).

Step 4 — Add Service Discovery

Goal - Remove hardcoded host:port dependencies.

Do - Add discovery-server. - Register gateway and services. - Switch gateway routes to lb://catalog-service and lb://order-service.

How to test - Restart services, verify gateway still routes correctly.

Exit criteria - No hardcoded downstream URLs; all routing uses service names.

Step 5 — Add Security (OAuth2/OIDC)

Goal - Secure APIs end-to-end.

Do - Start Keycloak locally (or your chosen IdP). - Configure gateway: - authenticate requests - Configure services: - validate JWT - enforce authorization via scopes/roles

How to test - Call without token → blocked. - Call with valid token → allowed.

Exit criteria - AuthN/AuthZ works at gateway and service layers.

Step 6 — Add Distributed Tracing

Goal - Trace a request across gateway → order-service → catalog-service.

Do - Enable Micrometer tracing + OTel export. - Ensure trace propagation for downstream calls.

How to test - Perform one request and verify single trace contains spans from all components.

Exit criteria - Traces visible in tracing backend and correlatable to logs.

Step 7 — Add Config Server

Goal - Centralize configuration.

Do - Add config-server. - Move common config from services into config repo. - Use profiles for local/dev/prod.

How to test - Change a config value and verify services pick it up after restart.

Exit criteria - Services boot using config-server managed configuration.

Step 8 — Local packaging and AWS readiness

Goal - Make the system portable and ready for AWS deployment.

Do - Containerize services. - Create docker-compose for local full-stack run.

Exit criteria - One command can bring up the full system locally.

Step 9 (later) — CI/CD + AWS deployment

Goal - Automated build/test/deploy.

Do - CI: build, unit tests, integration tests (Testcontainers for MySQL), static analysis. - CD: deploy to AWS (ECS Fargate or EKS) with RDS MySQL.

Exit criteria - You can deploy reliably on every merge.